

Part2: Fundamental of Recommender systems

Content Based Filtering

- 1. Content-based Filtering**
- 2. Types of Content-based Filtering**
- 3. How Content-based filtering works**
- 4. Approaches in Content-Based Filtering**
- 5. Feature Extraction with Example**
- 6. Advantages of Content-Based Filtering**
- 7. Challenges of Content-Based Filtering**

Content Filtering

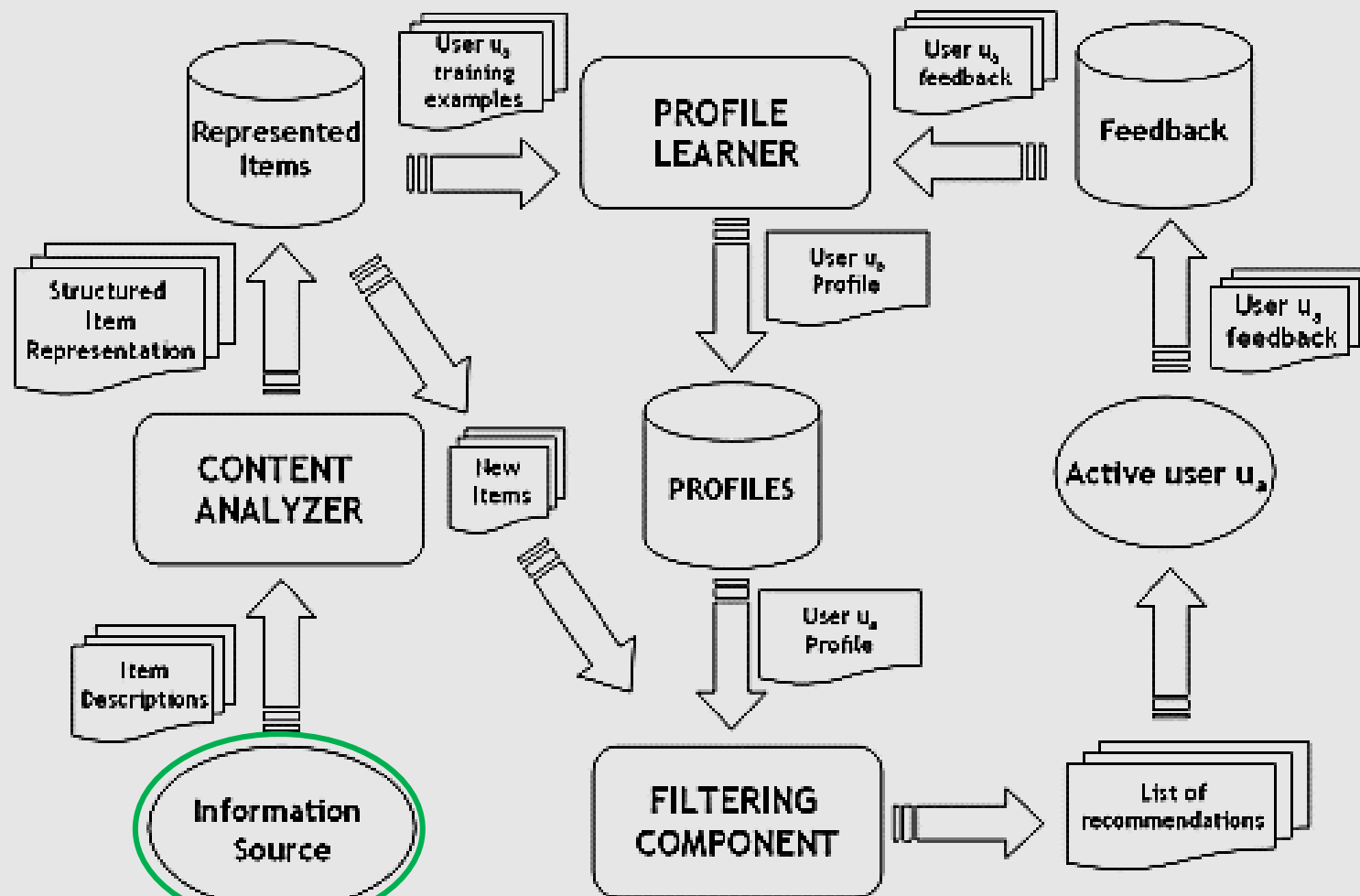
➤ Definition:

- Content-based filtering is a recommendation system technique that suggests items based on their attributes or features (Item profile: characteristics of the item such as the color of an object, authors of a book, and actors in a movie), comparing them to the user's preferences or past behavior.

➤ Key Concept:

- It uses the content or characteristics of the items themselves rather than relying on the preferences of other users.
- It focuses on understanding the content of items and mapping it to users' preferences

Content Filtering



Content-Based Filtering Overview

➤ How it works:

- **User Profiles:** Content-based systems create user profiles that summarize the items a user has interacted with (watched, read, liked, etc.) and build a representation of the features of those items.
- **Item Representation(description):** The system extracts features of items (e.g., keywords, genre, author, tags, etc.) and compares them to user preferences.

➤ Examples:

- **Music:** Suggesting songs based on genre, artist, or instrument.
- **Movies:** Recommending films based on actors, directors, genre, or plot keywords.

Approaches in Content-Based Filtering

➤ **Feature Extraction:**

- **Text-Based:** Involves text processing techniques like **TF-IDF** (Term Frequency-Inverse Document Frequency), **Word2Vec**, or **GloVe** (Global Vectors for Word Representation) to create item representations from text.
- **Image-Based:** For visual content, techniques like **CNNs** (**Convolutional Neural Networks**) can be used to extract visual features.
- **Audio-Based:** Features like **MFCC** (**Mel-Frequency Cepstral Coefficients**) or **spectrograms** for audio-based content (e.g., recommending podcasts or songs).

Approaches in Content-Based Filtering

➤ **User Profile Generation:**

- **Explicit Feedback:** Data based on what the user has rated or liked (e.g., movie ratings, clicks).
- **Implicit Feedback:** Data derived from user behavior (e.g., watch history, time spent on an article, etc.).

Approaches in Content-Based Filtering

➤ **Similarity Metrics:**

- **Cosine Similarity:** Measures the cosine of the angle between two vectors to determine similarity.
- **Euclidean Distance:** Measures the straight-line distance between two feature vectors.
- **Pearson Correlation:** Measures the linear correlation between two variables.
- **Dot product:** Measures the cosine of two vectors multiplied by each vector's projected length. It is best used for comparing item's with notably different magnitudes

Feature Extraction

➤ Text-based Traditional Approaches

- Bag of Words (BoW)
- TF-IDF (Term Frequency - Inverse Document Frequency)

Feature Extraction : Bag of Words (BoW)

- **BOW** is a common approach to represent items in content-based filtering is the **Bag of Words (BoW)** model. It represents each document (or item) simply as a vector based on the presence or absence of words, or sometimes based on the raw frequency of words
- **How BoW Works:**
 - **Step 1:** Build a vocabulary (set of all unique words) from the documents.
 - **Step2:** For each document, create a vector with the length equal to the vocabulary size, where each element of the vector represents the count of a specific word in that document.

Feature Extraction: Bag of Words (BoW)

➤ Example:

- Let's say we have 3 documents describing 3 different items (e.g., movies), and we want to compute the TF-IDF scores for terms in these documents.
- **Documents:**
 - **Doc1:** "action sci-fi alien spaceship"
 - **Doc2:** "romantic drama love story"
 - **Doc3:** "alien spaceship drama"

Feature Extraction: Bag of Words (BoW)

➤ Documents:

- **Doc1:** "action sci-fi alien spaceship"
- **Doc2:** "romantic drama love story"
- **Doc3:** "alien spaceship drama"



Vocabulary: {action, sci-fi, alien, spaceship, romantic, drama, love, story}

Feature Extraction: Bag of Words (BoW)

➤ Vocabulary: {action, sci-fi, alien, spaceship, romantic, drama, love, story}

➤ Documents:

•Doc1: "action sci-fi alien spaceship"

•Doc2: "romantic drama love story"

•Doc3: "alien spaceship drama"

Term	Doc1	Doc2	Doc3
action	1	0	0
sci-fi	1	0	0
alien	1	0	1
spaceship	1	0	1
romantic	0	1	0
drama	0	1	1
love	0	1	0
story	0	1	0

Feature Extraction: Bag of Words (BoW)

➤ The resulting vectors are:

✓ **Doc1:** [1, 1, 1, 1, 0, 0, 0, 0]

✓ **Doc2:** [0, 0, 0, 0, 1, 1, 1, 1]

✓ **Doc3:** [0, 0, 1, 1, 0, 1, 0, 0]



Term	Doc1	Doc2	Doc3
action	1	0	0
sci-fi	1	0	0
alien	1	0	1
spaceship	1	0	1
romantic	0	1	0
drama	0	1	1
love	0	1	0
story	0	1	0

Feature Extraction : TF-IDF

- **TF-IDF** is a statistical measure used to evaluate how important a word is to a document in a collection (or corpus).
- It is often used in content-based filtering to represent items, especially when items are described by textual information (e.g., movies, articles, products).
- It helps to weigh down the importance of common words that appear frequently across many items and boost the significance of words that are more unique to a specific item.

Feature Extraction : TF-IDF

- **Term Frequency (TF):** Measures how frequently a term occurs in a document. If a term appears multiple times in a document, it gets a higher score.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- **Inverse Document Frequency (IDF):** Measures how unique or rare a term is across the whole collection of documents. If a term appears in many documents, its IDF score will be low.

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing term } t} \right)$$

- **TF-IDF Score:** The final score for each term in a document is the product of its TF and IDF values.

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Feature Extraction : TF-IDF

- Based on the previous example:
- Step 1: Calculate Term Frequency (TF)

Term	Doc1	Doc2	Doc3
action	(1/4)	0	0
sci-fi	(1/4)	0	0
alien	(1/4)	0	(1/3)
spaceship	(1/4)	0	(1/3)
romantic	0	(1/4)	0
drama	0	(1/4)	(1/3)
love	0	(1/4)	0
story	0	(1/4)	0

Feature Extraction : TF-IDF

- **Step 2: Calculate Inverse Document Frequency (IDF)**
- Total number of documents = 3.
- The number of documents containing each term (df):

Term	df	IDF
action	1	$\log(3/1) = 1.0986$
sci-fi	1	$\log(3/1) = 1.0986$
alien	2	$\log(3/2) = 0.4055$
spaceship	2	$\log(3/2) = 0.4055$
romantic	1	$\log(3/1) = 1.0986$
drama	2	$\log(3/2) = 0.4055$
love	1	$\log(3/1) = 1.0986$
story	1	$\log(3/1) = 1.0986$

Feature Extraction : TF-IDF

- **Step 3: Calculate TF-IDF Score**
- Multiply TF by IDF for each term in each document:

Term	Doc1 (TF-IDF)	Doc2 (TF-IDF)	Doc3 (TF-IDF)
action	$(1/4) * 1.0986$	0	0
sci-fi	$(1/4) * 1.0986$	0	0
alien	$(1/4) * 0.4055$	0	$(1/3) * 0.4055$
spaceship	$(1/4) * 0.4055$	0	$(1/3) * 0.4055$
romantic	0	$(1/4) * 1.0986$	0
drama	0	$(1/4) * 0.4055$	$(1/3) * 0.4055$
love	0	$(1/4) * 1.0986$	0
story	0	$(1/4) * 1.0986$	0

Feature Extraction : TF-IDF

- **Step 3: Calculate TF-IDF Score**
- Multiply TF by IDF for each term in each document where Doc1(Movie 1), Doc2(Movie 2) and so on :

Term	Movie 1 (TF-IDF)	Movie 2 (TF-IDF)	Movie 3 (TF-IDF)
action	$1/4 \times 1.0986 = 0.2746$	0	0
sci-fi	$1/4 \times 1.0986 = 0.2746$	0	0
alien	$1/4 \times 0.4055 = 0.1014$	0	$1/3 \times 0.4055 = 0.1352$
spaceship	$1/4 \times 0.4055 = 0.1014$	0	$1/3 \times 0.4055 = 0.1352$
romantic	0	$1/4 \times 1.0986 = 0.2746$	0
drama	0	$1/4 \times 0.4055 = 0.1014$	$1/3 \times 0.4055 = 0.1352$
love	0	$1/4 \times 1.0986 = 0.2746$	0
story	0	$1/4 \times 1.0986 = 0.2746$	0

Feature Extraction : TF-IDF

- Thus, each document is represented by a vector where each term's value is its TF-IDF score.
- For example:
 - **Doc1 vector:** [0.2746, 0.2746, 0.1014, 0.1014, 0, 0, 0, 0]
 - **Doc2 vector:** [0, 0, 0, 0, 0.2746, 0.1014, 0.2746, 0.2746]
 - These vectors can then be compared with user preferences or item profiles to find similar items based on the TF-IDF values of their descriptions.

Similarity

Now that we have the **TF-IDF vectors** for each movie, we can calculate the **cosine similarity** between Movie 1 and the other movies. The formula for cosine similarity between two vectors Doc1(Movie 1) and Doc2(Movie 2) is:

Term	Movie 1 (TF-IDF)	Movie 2 (TF-IDF)	Movie 3 (TF-IDF)
action	$1/4 \times 1.0986 = 0.2746$	0	0
sci-fi	$1/4 \times 1.0986 = 0.2746$	0	0
alien	$1/4 \times 0.4055 = 0.1014$	0	$1/3 \times 0.4055 = 0.1352$
spaceship	$1/4 \times 0.4055 = 0.1014$	0	$1/3 \times 0.4055 = 0.1352$
romantic	0	$1/4 \times 1.0986 = 0.2746$	0
drama	0	$1/4 \times 0.4055 = 0.1014$	$1/3 \times 0.4055 = 0.1352$
love	0	$1/4 \times 1.0986 = 0.2746$	0
story	0	$1/4 \times 1.0986 = 0.2746$	0

Similarity

- Now that we have the **TF-IDF vectors** for each movie, we can calculate the **cosine similarity** between Movie 1 and the other movies. The formula for cosine similarity between two vectors Doc1(Movie 1) and Doc2(Movie 2) is:
- **TF-IDF Vectors:**
 - ✓ **Movie 1:** [0.2746,0.2746,0.1014,0.1014,0,0,0,0]
 - ✓ **Movie 2:** [0,0,0,0,0.2746,0.1014,0.2746,0.2746]
 - ✓ **Movie 3:** [0,0,0.1352,0.1352,0,0.1352,0,0]

Similarity

➤ Cosine Similarity between Movie 1 and Movie 2

✓ **Movie 1:** [0.2746,0.2746,0.1014,0.1014,0,0,0,0]

✓ **Movie 2:** [0,0,0,0,0.2746,0.1014,0.2746,0.2746]

1. **Dot Product** of Movie 1 and Movie 2:

$$(0.2746 \times 0) + (0.2746 \times 0) + (0.1014 \times 0) + (0.1014 \times 0) + (0 \times 0.2746) + (0 \times 0.1014) + (0 \times 0.2746) + (0 \times 0.2746) = 0$$

2. **Magnitude** of Movie 1:

$$\sqrt{(0.2746)^2 + (0.2746)^2 + (0.1014)^2 + (0.1014)^2} = \sqrt{0.0754} = 0.2746$$

3. **Magnitude** of Movie 2:

$$\sqrt{(0.2746)^2 + (0.1014)^2 + (0.2746)^2 + (0.2746)^2} = \sqrt{0.2548} = 0.5048$$

4. **Cosine Similarity** between Movie 1 and Movie 2:

$$\frac{0}{0.2746 \times 0.5048} = 0$$

Similarity

➤ Cosine Similarity between Movie 1 and Movie 3

✓ **Movie 1:** [0.2746,0.2746,0.1014,0.1014,0,0,0,0]

✓ **Movie 3:** [0,0,0.1352,0.1352,0,0.1352,0,0]

Based on these cosine similarity scores, **Movie 3** is more similar to **Movie 1** than **Movie 2**, and would be recommended to the user.

1. **Dot Product** of Movie 1 and Movie 3:

$$(0.2746 \times 0) + (0.2746 \times 0) + (0.1014 \times 0.1352) + (0.1014 \times 0.1352) = 0.0274$$

2. **Magnitude** of Movie 1: Already calculated as 0.2746.

3. **Magnitude** of Movie 3:

$$\sqrt{(0.1352)^2 + (0.1352)^2 + (0.1352)^2} = \sqrt{0.0548} = 0.2342$$

4. **Cosine Similarity** between Movie 1 and Movie 3:

$$\frac{0.0274}{0.2746 \times 0.2342} = \frac{0.0274}{0.0643} = 0.4262$$

Feature Extraction

➤ TF-IDF vs BoW

- **TF-IDF** gives more weight to less frequent words, which can capture the importance of specific terms in items and reduce the influence of common words.
 - **BoW** is simpler and only captures the raw frequency of words or their presence without considering their overall importance in the corpus.
- Both methods are widely used in content-based filtering, depending on the dataset and problem at hand.

Advantages of Content-Based Filtering

- ✓ **Personalized Recommendations:** Recommendations are tailored to the user based on their previous actions or preferences.
- ✓ **No Cold Start Problem for Users:** Unlike collaborative filtering, content-based systems do not need other users' data, so the cold start problem for new users is not an issue.
- ✓ **Transparency:** Users can understand why a particular item is recommended based on its features.

Challenges of Content-Based Filtering

- **Limited Serendipity:** Recommendations might be too similar to what the user has already seen or liked, leading to a lack of diversity in suggestions.
- **Feature Engineering:** Extracting meaningful features that truly represent items can be complex, especially for unstructured data like text or images.
- **Overfitting:** Recommendations might become narrow if user profiles are too rigid and do not evolve over time.